

Documentazione Progetto InfoLab

Giovanni Menicucci

June 2026

1 Introduzione

Questo documento tiene traccia dello sviluppo del progetto finale per il corso di Informatica con Laboratorio 2025/26, riguardante la soluzione dell'equazione del calore 2D.

2 Struttura del progetto

I task richiesti nel progetto vengono eseguiti con codici C++ (reperibili nella cartella "codice.cpp" o Python (nella cartella "codice.python"). Alcuni codici producono file di testo (nella cartella "dati"), che poi vengono letti dai codici successivi.

Le sezioni seguenti contengono i moduli principali che compongono il progetto.

3 Discretizzazione del dominio

Input: numero naturale N . Il dominio $[0, 1]^2$ è discretizzato con una griglia equispaziata di $(N + 2) \times (N + 2)$ punti.

Output: grafo di adiacenza, i cui nodi sono i punti interni alla griglia, con archi tra nodi primi vicini nella griglia.

File coinvolti: codice `discretizzazione.cpp` che genera un file `coords.txt` che rappresenta la griglia e un file `connectivity.txt` che rappresenta il grafo.

Strutture dati necessarie:

- **struct** per i nodi, per memorizzare indice progressivo n , indici della griglia i, j , coordinate spaziali x, y .
- **vector** per contenere la sequenza dei nodi interni, ordinati secondo l'indice n , per poi esportarli su `coords.txt`.
- **vector** di **vector** per la lista di adiacenza del grafo (lista e non matrice perché il grafo è sparso); e anche per costruire una matrice **griglia** che associ alle coordinate spaziali di ciascun nodo interno il relativo indice progressivo, e associ -1 ai nodi di bordo.

Funzioni:

- **GeneraGriglia**: calcola il passo della griglia e genera i singoli nodi interni con un doppio ciclo for che assegna indice progressivo n (ordine lessicografico), coordinate nella griglia e spaziali.

Riempie il vettore dei nodi interni e la matrice `griglia`, aumentando n di 1 alla fine di ogni iterazione. La matrice è inizializzata nel `main` con valori -1 . Complessità: $O(N^2)$, a causa del doppio ciclo `for`. All'interno di ogni iterazione si fanno solo operazioni elementari.

- **EsportaPuntiInterni**: crea il file `coordsN.txt`, scorre sul vettore dei nodi interni e li scrive sul file, usando una riga per ciascun nodo, contenente n, i, j, x, y . Complessità: $O(N^2)$ (lunghezza del vettore su cui scorro).
- **GeneraEsportaGrafo**. Crea il file `connectivityN.txt`. Svolge un doppio ciclo `for`, sugli indici dei nodi interni. Ad ogni iterazione, usa la matrice `griglia` per verificare se i 2 vicini a destra e in alto del nodo corrente sono di bordo o no (se un nodo è di bordo, il valore assegnatogli nella matrice è rimasto -1 , cioè non è stato sovrascritto con l'indice progressivo). Controlla solo i vicini a destra e in alto, perché se controllassi tutti e 4 conterei due volte gli archi. Se il vicino non è di bordo, aggiunge alla lista di adiacenza (vector di N vector, inizializzato nel `main`) i due nodi coinvolti. Poi scrive in una riga del file l'indice progressivo e dell'arco (cioè quanti archi ho creato fino a questo) e gli indici progressivi dei due nodi. Complessità: $O(N^2)$, a causa del doppio ciclo `for`.

Complessità totale: $O(N^2)$.

4 Riordinamento dei nodi

Input: grafo definito sopra.

Output: nodi del grafo riordinati.

File coinvolti: codice `riordinamento.cpp` che prende i file `coords.txt` e `connectivity.txt` generati nel modulo precedente, e genera un file `ordering.txt` che rappresenta la permutazione dei nodi.

Strutture dati necessarie:

- **struct** per i nodi, come nel modulo di discretizzazione.
- **vector** per le sequenze di nodi.

Funzioni:

- Generazione vettore dei nodi: dovrebbe leggere il file `coords.txt` e riempire un vettore con i nodi. In pratica inverte quanto fatto nella funzione `EsportaPuntiInterni` del modulo di discretizzazione. Complessità attesa: $O(N^2)$.
- Riordinamento dei nodi: dovrebbe prendere il vettore dei nodi e applicare la nested dissection, riempiendo un nuovo vettore con i nodi nell'ordine finale. Per farlo uso la ricorsione. Scorro sul vettore dei nodi, assegnandoli ai tre insiemi V_1, V_2, V_S (vector) definiti nel testo, e poi applico la funzione a V_1 e V_2 . Tra gli argomenti della funzione dovranno esserci anche gli indici i, j dei nodi contenuti nel vettore, per decidere se dividere verticalmente o orizzontalmente in base a quale lato del rettangolo è maggiore. Il caso base della ricorsione sarà quando il rettangolo non è divisibile ulteriormente nei tre sottoinsiemi. Infine dovrò concatenare i vettori V_1, V_2, V_S , la loro unione sarà l'ordinamento finale dei nodi e sarà l'output della funzione. Complessità attesa: $O(N^2 \log N)$, perché ad ogni passo della ricorsione devo scorrere sul vettore dei nodi corrente, che ha lunghezza $O(N^2)$, e ad ogni passo della ricorsione divido per 2 (circa) il lato del rettangolo considerato, da cui il $\log N$.
- Esportazione su file: dovrebbe creare il file `ordering.txt` e scorrere sul vettore generato dalla funzione sopra, con i nodi nell'ordine finale, scrivendo per ciascun nodo in una riga la posizione m nel vettore e l'indice progressivo iniziale n contenuto nella struct. Complessità attesa: $O(N^2)$, che è la lunghezza del vettore.

Complessità totale attesa: $O(N^2 \log N)$.

5 Approssimazione dell'equazione differenziale con sistema lineare

Input: dominio dell'equazione differenziale discretizzato (ordinato nei due modi), coefficienti, funzione sorgente, condizione al bordo.

Output: matrice e termine noto del sistema lineare che rappresenta l'equazione.

Implementazione: codice `approx_lineare.cpp` che prende i file `coords.txt` (generato nel modulo "discretizzazione") e `ordering.txt` (generato nel modulo "riordinamento") e genera un file `A.txt` che rappresenta la matrice del sistema lineare, e un file `rhs.txt` che rappresenta il termine noto.

6 Fattorizzazione di Cholesky

Input: matrice A del sistema lineare.

Output: matrice L che fattorizza A . Immagini della struttura di A, L (entrate non nulle).

Implementazione: notebook `.ipynb` che legge `A.txt` generato nel modulo precedente ed esegue la fattorizzazione. Struttura della matrice (entrate non nulle).

7 Soluzione del sistema lineare

Input: fattorizzazione di A e termine noto del sistema lineare.

Output: soluzione del sistema lineare. Analisi del tempo necessario al variare di N e dell'ordinamento dei nodi. Plot della soluzione.

Implementazione: notebook `.ipynb` che legge la matrice L e `rhs.txt` e risolve il sistema lineare.